

PSYC 51.17: Models of Language and Communication - Week 4

Contextual Embeddings: ELMo, USE, BERT

Lecture 12: Beyond Static Word Representations

PSYC 51.17: Models of Language and Communication - Week 4

Winter 2026

Winter 2026

Today's Lecture

1. **From Static to Contextual**
2. **Language Models as Feature Extractors**
3. **ELMo: Embeddings from Language Models**
4. **Universal Sentence Encoder**
5. **BERT: Bidirectional Transformers**
6. **Comparison & Applications**

*Goal: Understand how context transforms word
representation*

Winter 2026

The Polysemy Problem Revisited

Recall: Static embeddings assign ONE vector per word

Example: "bank"

1. "I deposited money at the **bank**"
(financial institution)
2. "We sat by the river **bank**"
(river bank)

Cosine Similarity Demo:

```
1 # Static
  embeddings
  (Word2Vec) 3
2 sim(bank_sent1,
  bank_sent2) =
```

Static vs. Contextual Embeddings

Static (Word2Vec, GloVe, FastText):

```
1 # Same vector every time
2 model = Word2Vec(...)
3 vec1 = model['bank']
4 vec2 = model['bank']
5
6 assert vec1 == vec2 # True!
```

Characteristics:

- One vector per word type
- Context-independent
- Fast lookup (dictionary)
- Fixed after training

Contextual (ELMo, BERT):

```
1 # Different vector per
2   occurrence
3   model =
4     BertModel.from_pretrained('bert-
5     base')
6
7   sent1 = "river bank"
8   sent2 = "money bank"
9
10  vec1 = get_embedding(model,
11    sent1, 'bank')
12  vec2 = get_embedding(model,
13    sent2, 'bank')
14
15  assert vec1 != vec2 # True!
```

Language Models as Feature Extractors

Key Insight: Train a language model, use its internal states as embeddings

Language Modeling Task:

Predict the next word given previous words:

$$P(w_t | w_1, w_2, \dots, w_{t-1})$$

Worked Example

Input: "The cat sat on the ____"

Model predicts probabilities:	
-------------------------------	--

ELMo: Embeddings from Language Models

The first widely-adopted contextual embedding
(2018)

Key Ideas:

1. Train deep bidirectional language model
2. Use all layer activations
3. Weighted combination per

Bidirectional Processing:

1	Forward: The → cat → sat → ...	6
2	Backward: ← sat ← cat ←	

ELMo: How It Works

Training:

1. Pre-train on large corpus (1B Word Benchmark)
2. Each position gets representation from all layers

Usage (downstream tasks):

1. Freeze ELMo weights
2. For each token, extract representations from all layers
3. Learn task-specific weighted combination

ELMo in Practice

```
1 from allennlp.modules.elmo import Elmo, batch_to_ids
2
3 # Initialize ELMo
4 options_file = "https://s3-us-west-
5 2.amazonaws.com/allennlp/models/elmo/2x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2x
6
7 weight_file = "https://s3-us-west-
8 2.amazonaws.com/allennlp/models/elmo/2x4096_512_2048cnn_2xhighway/elmo_2x4096_512_2048cnn_2x
9
10 elmo = Elmo(options_file, weight_file, 2, dropout=0)
11
12 # Prepare sentences
13 sentences = [
14     ['I', 'deposited', 'money', 'at', 'the', 'bank'],
15     ['We', 'sat', 'by', 'the', 'river', 'bank']
16 ]
```

Winter 2026

8

continued...

ELMo in Practice

```
15 # Convert to character ids
16 character_ids = batch_to_ids(sentences)
17
18 # Get embeddings
19 embeddings = elmo(character_ids)
20
21 # embeddings['elmo_representations'] contains:
22 # - List of 2 tensors (one per layer)
23 # - Shape: [batch_size, seq_len, 1024]
24
25 # Different vectors for "bank"!
26 bank1 = embeddings['elmo_representations'][0][0, 5, :] # first sentence
27 bank2 = embeddings['elmo_representations'][0][1, 5, :] # second sentence
28
```

Winter 2026

ELMo in Practice

29 | # Cosine similarity will be lower than for static embeddings

Winter 2026

Universal Sentence Encoder (USE)

Sentence-level embeddings for semantic similarity

Motivation:

- Word embeddings: good for words
- But what about sentences?
- Average of word vectors? Too simple!
- Need compositionality

Two Variants:

1. Transformer-based:

- Higher accuracy
- Slower (compute intensive)

Training Objectives:

1. Unsupervised: Skip-thought
2. Supervised: SNLI (entailment)
3. Multi-task learning

Output:

- 512-dimensional vector
- Fixed-length (any sentence length)
- Optimized for similarity tasks

Universal Sentence Encoder in Practice

```
1 import tensorflow_hub as hub
2 import numpy as np
3
4 # Load model
5 embed = hub.load("https://tfhub.dev/google/universal-sentence-encoder/4")
6
7 # Example sentences
8 sentences = [
9     "The cat sat on the mat.",
10    "A feline rested on the rug.",
11    "The dog ran in the park.",
12    "I love machine learning."
13 ]
14
15 # Generate embeddings
```

Universal Sentence Encoder in Practice

```
16 embeddings = embed(sentences)
17
18 # Shape: [4, 512]
19 print(embeddings.shape)
20
21 # Compute similarity
22 from sklearn.metrics.pairwise import cosine_similarity
23
24 sim_matrix = cosine_similarity(embeddings)
25 print(sim_matrix)
26
27 # Sentences 1 and 2 should be very similar (paraphrases)
28 # Sentence 3 somewhat similar (animals)
29 # Sentence 4 dissimilar
30
```

Winter 2026

13

...continued...

Universal Sentence Encoder in Practice

```
31 # Use for semantic search
32 query = "cat on mat"
33 query_embedding = embed([query])
34 similarities = cosine_similarity(query_embedding, embeddings)[0]
35 most_similar_idx = np.argmax(similarities)
36 print(f"Most similar: {sentences[most_similar_idx]}")
```

Winter 2026

BERT: Bidirectional Encoder Representations

The model that changed everything (2018)

Key Innovations:

1. context (not just left-to-right)
2. architecture (attention)
3. pre-training
4. Deeply bidirectional

Architecture Sizes:

BERT- Large	24	1024
------------------------	-----------	-------------

Training Data:

15

- BooksCorpus (800M words)

Masked Language Modeling (MLM)

BERT's key training innovation

The Problem with Traditional LM:

- Left-to-right: Only sees previous words
- Right-to-left: Only sees next words
- Want: See both directions simultaneously
- But: Can't just show the answer during training!

Solution: Mask some words, predict them

Worked Example: MLM Training

Original: "The cat sat on the mat"

Step 1: Randomly select 15% of tokens → "cat" selected

Next Sentence Prediction (NSP)

Second pre-training task: Understand sentence relationships

Task: Given two sentences A and B, predict if B follows A in the text

Positive Example (IsNext)

Sentence A: "The cat sat on the mat."

Sentence B: "It was sleeping peacefully."

Label: IsNext

Negative Example (NotNext)

Sentence A: "The cat sat on the mat."

Sentence B: "Machine learning is fascinating."

BERT Architecture

Three types of embeddings are summed for each token:

Worked Example: Input Representation

Input: "[CLS] I love NLP [SEP] It is fun [SEP]"

Token	Token ID	Segment	Position	Final Embedding
[CLS]	E_CLS	A	0	$E_{CLS} + E_A + E_0$
I	E_I	A	1	$E_I + E_A + E_1$
love	E_love	A	2	$E_{love} + E_A + E_2$
NLP	E_NLP	A	3	$E_{NLP} + E_A + E_3$
[SEP]	E_SEP	A	4	$E_{SEP} + E_A + E_4$
It	E_It	B	5	$E_{It} + E_B + E_5$
is	E_is	B	6	$E_{is} + E_B + E_6$

BERT Architecture

Token	Token ID	Segment	Position	Final Embedding
[SEP]	E_SEP	B	8	$E_SEP + E_B + E_8$

- **Token Embedding:** What word is this?
- **Segment Embedding:** Which sentence (A or B)?
- **Position Embedding:** Where in the sequence?

Winter 2026

BERT in Practice

```
1 from transformers import BertTokenizer, BertModel
2 import torch
3
4 # Load pre-trained BERT
5 tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
6 model = BertModel.from_pretrained('bert-base-uncased')
7
8 # Example sentences with "bank"
9 sent1 = "I deposited money at the bank"
10 sent2 = "We sat by the river bank"
11
12 # Tokenize
13 tokens1 = tokenizer(sent1, return_tensors='pt')
```


BERT in Practice

```
14 tokens2 = tokenizer(sent2, return_tensors='pt')
15
16 # Get embeddings
17 with torch.no_grad():
18     output1 = model(**tokens1)
19     output2 = model(**tokens2)
20
21 # Last hidden state: [batch_size, seq_len, hidden_size]
22 embeddings1 = output1.last_hidden_state
23 embeddings2 = output2.last_hidden_state
24
25 # Extract "bank" embedding (position varies)
26 # tokens1: [CLS] i deposited money at the bank [SEP]
```

Winter 2026

21

...continued...

BERT in Practice

```
27 bank1_embedding = embeddings1[0, 6, :] # 768-dim vector
28
29 # tokens2: [CLS] we sat by the river bank [SEP]
30 bank2_embedding = embeddings2[0, 6, :] # 768-dim vector
31
32 # Different vectors for "bank"!
33 from torch.nn.functional import cosine_similarity
34 sim = cosine_similarity(bank1_embedding, bank2_embedding, dim=0)
35 print(f"Similarity: {sim:.3f}") # Lower than with static embeddings
```

Winter 2026

Fine-tuning BERT

Two ways to use BERT:

1. Feature Extraction:

- Freeze BERT weights
- Use embeddings as features
- Train classifier on top
- Faster, less data needed

```
1 # Freeze BERT
2 for param in
  bert_model.parameters():
3   param.requires_grad = False
4
5 # Add classifier
```

2. Fine-tuning:

- Update BERT weights
- Add task-specific head
- Train end-to-end
- Better performance, more data needed

```
1 # Keep BERT trainable
2 bert_model =
3   BertModel.from_pretrained(
4     'bert-base-uncased'
5   )
```

Fine-tuning BERT

```
12 | list(classifier.parameters()),  
13 | lr=2e-5 # Small learning rate!  
14 | )
```

Best Practice: Fine-tune with small learning rate ($1e-5$ to $5e-5$) for few epochs (2-4)

Winter 2026

Contextual Embeddings Comparison

Pre-training	LM (forward+backward)	Multi-task	MLM + NSP
Bidirectional	Shallow	Yes	Deep
Granularity	Token	Sentence	Token
Hidden size	1024	512	768/1024
Parameters	93M	256M	110M/340M
Speed	Medium	Fast	Slow
OOV handling	Characters	Subwords	WordPiece
Year	2018	2018	2018

Recommendations

- **ELMo**: Legacy systems, character-aware needs
- **USE**: Sentence similarity, semantic search, fast inference
- **BERT**: State-of-the-art quality, most NLP tasks (now superseded)

Impact on NLP

BERT revolutionized NLP:

Before BERT (pre-2018):

- Task-specific architectures
- Train from scratch
- Static embeddings (Word2Vec, GloVe)
- Limited transfer learning
- Moderate performance

After BERT (post-2018):

- Pre-train, then fine-tune
- Transfer learning standard
- Contextual embeddings
- Massive pre-trained models
- State-of-the-art results

Tasks that improved:

- Question Answering (+1.5 F1 on SQuAD)
- NER (+0.3 F1)

BERT Variants:

- RoBERTa: Better training
- ALBERT: Parameter sharing

Real-World Applications

1. Google Search:

```
1 Query: "can you get
2   medicine for
3   someone pharmacy"
4 BERT understands: picking
5   up a
6   prescription FOR someone
7   else
8 Before BERT: matched
   "medicine"
   and "pharmacy" keywords
```

3. Sentiment Analysis:

```
1 # Fine-tuned BERT
2 text = "Not bad at all!"
3 prediction = model(text)
4 # → Positive (understands
   negation!)
```

4. Named Entity Recognition:

```
1 Input: "Apple CEO Tim Cook27
2 announced ... "
3 Output:
```

Discussion Question

Do contextual embeddings truly "understand" language?

Consider:

- BERT can distinguish "bank" (financial) from "bank" (riverside)
- It achieves human-level performance on many benchmarks
- But it's trained only on text co-occurrence patterns

Arguments For:

- Captures complex semantic relationships
- Generalizes to new contexts
- Emergent linguistic capabilities
- Handles compositional meaning

Arguments Against:

- No grounding in physical world
- No common sense reasoning
- Exploits statistical shortcuts
- Brittle to adversarial examples

2. Fine-tuning Best Practices:

- Small learning rate ($2e-5$ typical)
- Few epochs (2-4)
- Batch size: 16 or 32
- Warm-up steps
- Gradient clipping

3. Computational Considerations:

- BERT-base: ~110M params, 512 max tokens
- Needs GPU (1-4 GB VRAM minimum)
- Batching for efficiency

3. Universal Sentence Encoder (2018):

- Sentence-level embeddings
- Two variants: Transformer & DAN
- Optimized for semantic similarity

4. BERT (2018):

BERT Variants:

- Liu et al. (2019). "RoBERTa: A Robustly Optimized BERT Pretraining Approach"
- Sanh et al. (2019). "DistilBERT, a distilled version of BERT"
- Lan et al. (2019). "ALBERT: A Lite BERT for Self-supervised Learning"

Critical Perspectives:

- Bender & Koller (2020). "Climbing towards NLU: On

Questions?

Next Lecture:

Dimensionality Reduction: PCA, t-SNE, UMAP

Visualizing high-dimensional embeddings!

Winter 2026